深度強化學習 HW1

網格世界策略評估與價值迭代

Grid World Policy Evaluation & Value Iteration

AI 輔助開發對話記錄

2026.03.12

GitHub: https://github.com/albertiscool/hw1

Demo: https://albertiscool.github.io/hw1/

Phase 1 — 網格地圖建置與策略評估實作

User

我需要基於作業規格實作 HW1 的網格世界系統。
HW1-1 要求使用 Flask 作為後端，建構一個 n x n 的互動式網格地圖（n 介於 5~10），
讓使用者透過點擊設定起點、終點與 n-2 個障礙物。
HW1-2 則需要實作隨機策略生成與 Policy Evaluation，
並將 Value Matrix 和 Policy Matrix 視覺化呈現在前端。
請根據這份作業規格進行實作。

Assistant

分析作業需求後，完成以下實作：

[app.py — Flask 後端]
- 路由 /evaluate：接收前端透過 AJAX 傳來的網格設定
- 隨機生成策略後執行 Policy Evaluation（Bellman Equation）
- 參數：gamma=0.9, reward_step=-1, reward_goal=10, theta=1e-6

[index.html — 前端介面]
- 動態生成 n x n 網格，支援點擊設定起點(綠)/終點(紅)/障礙物(灰)
- 障礙物支援 toggle 切換（可取消再重選）
- 步驟引導提示使用者當前操作階段
- AJAX 非同步呼叫後端，渲染 Value Matrix 與 Policy Matrix

圖 1：5x5 網格設定 — 起點(S)、終點(E)、障礙物

Phase 2 — 策略評估演算法分析與驗證

User

我想確認目前 Policy Evaluation 的實作是否符合理論基礎。
根據課程教材，PE 應該使用 Bellman Equation 作為核心運算，
而且演算法比較表中提到 PE 使用「動作機率加權和」，
VI 使用「Max 運算」，策略萃取使用「Argmax」。
目前的程式是否有正確對應到這些概念？
另外 HW1-3 的價值迭代部分目前尚未實作，也需要補上。

Assistant

逐項對照分析：

[已正確實作 — Bellman 方程式]
V(s) = R(s,a) + gamma * V(s')
對應教材「階段二」的核心運算。

[演算法比較]
- 策略評估 (PE)：動作機率加權和 (Sum) -> 估算給定策略的價值
- 價值迭代 (VI)：取最大動作價值 (Max) -> 直接找最佳價值函數
- 策略萃取：Argmax 運算 -> 決定狀態的最終最佳動作

[待實作 — HW1-3 階段三]
- 價值迭代需引入 Max 運算
- 策略萃取需使用 Argmax 提取確定性最佳動作

Phase 3 — 價值迭代與策略萃取實作

User

請接著實作 HW1-3 的部分，包含 Value Iteration 和 Policy Extraction。
另外，目前的 UI 風格過於花俏，與作業範例的簡潔風格不符，
請調整為白色背景的基本排版，貼近作業規格中的示意圖。

Assistant

[A. UI 調整]
移除深色漸層與動畫效果，改為白色背景、基本按鈕樣式，
簡潔的表格邊框，貼合作業範例截圖的風格。

[B. 新增 /value_iteration 路由]
價值迭代核心公式：
V(s) = max_a [R(s,a) + gamma * V(s')]

[C. 策略萃取]
pi*(s) = argmax_a [R(s,a) + gamma * V*(s')]
從收斂的 V* 中對每個狀態取最佳動作。

[D. 最佳路徑繪製]
從起點沿最佳策略走到終點，萃取路徑並以黃色標示。

[效能比較]
PE 需要約 120+ 次迭代收斂，VI 僅需約 9 次即收斂，
體現了 Max 運算在收斂效率上的顯著優勢。

圖 2：HW1-2 策略評估 vs HW1-3 價值迭代結果比較

Phase 4 — 策略評估修正：確定性 → 隨機策略

User

執行 Policy Evaluation 後發現 Value Matrix 所有格子的值都是 -10.00，
和作業範例中各格子應呈現不同數值的結果不一致。
我推測問題出在策略的機率模型上——
目前似乎是對每個狀態只指定單一確定性動作，
而非教材中提到的「動作機率加權和」。
能否確認這個假設並修正實作？

Assistant

[問題診斷]
確認使用者的推測正確。原實作為每個狀態隨機指定「一個」確定性動作，
導致大部分路徑形成循環而永遠無法到達終點。
數學上收斂到 V = -1 / (1 - 0.9) = -10（每步 reward=-1 的無限折扣和）。

[修正方案]
改用 Stochastic Policy，每個動作機率 pi(a|s) = 0.25：
V(s) = Sum_a pi(a|s) * [R(s,a) + gamma * V(s')]
= 0.25*[R+gV(up)] + 0.25*[R+gV(down)] + 0.25*[R+gV(left)] + 0.25*[R+gV(right)]

[修正結果]
各格子呈現不同數值：
- 起點附近 = -9.71（離終點遠）
- 終點旁邊 = -0.17（離終點近）
值的分佈反映了各狀態到終點的「期望折扣距離」，與理論一致。

關鍵差異整理：
X 確定性策略：每狀態只考慮 1 個動作 -> 路徑循環 -> 全部 -10
O 隨機策略(Stochastic)：4 個動作各 25% 加權 -> 部分機率到達終點 -> 值因距離而異

圖 3：修正後的 Policy Evaluation — 各格子呈現不同價值

Phase 5 — GitHub Pages 靜態部署

User

我已經在 GitHub 上建立了 GitHub Pages，部署網址為：
https://albertiscool.github.io/hw1/
希望能將網格地圖的互動介面部署上去，
讓使用者可以不需要架設 Flask 伺服器也能直接使用。

Assistant

[問題分析]
GitHub Pages 僅支援靜態檔案（HTML/CSS/JS），無法執行 Python/Flask 後端。
因此需要將所有後端運算邏輯移植到前端 JavaScript。

[實作方案]
在專案根目錄建立獨立的 index.html，將以下 Python 邏輯改寫為 JavaScript：
- Policy Evaluation（隨機策略的加權求和）
- Value Iteration（Max 運算）
- Policy Extraction（Argmax 運算）
- Path Extraction（沿最佳策略的路徑萃取）

[部署結果]
推送後 GitHub Pages 自動讀取根目錄 index.html，
所有運算在瀏覽器端完成，網站即時可用：
https://albertiscool.github.io/hw1/

架構差異：
Flask 版：前端 (HTML) <-> AJAX <-> 後端 (Python) — 需啟動伺服器
靜態版：前端 (HTML + JS) — 瀏覽器直接運算，無需伺服器